

TRPO

LAMDA-5

May 27, 2026

我们这次的目标是实现算法 TRPO (**Trust Region Policy Optimization**, <https://arxiv.org/pdf/1502.05477>), 是 2015 年由当时在 UC Berkeley 读博的 John Schulman 及其导师 Pieter Abbeel 等人所提出, 发表在 JMLR, 目前引用量已过万。它成功地将传统运筹学中的“信赖域”方法引入深度强化学习, 首次在深度神经网络模型上为策略优化提供了“单调改进”的数学保证, 这意味着原则上每次更新都会让策略变得更好, 不会出现灾难性退步。

1 代码补全

该项目的实现参考了<https://spinningup.openai.com/en/latest/algorithms/trpo.html>, 请补全文件 `algorithm/trpo.py` 中的 **TODO 部分** (完善函数 `compute_advantages_from_traj`, `compute_advantages_from_rollout`, 以及 `_update_policy`, 相关功能要求已在注释中标出)。并回答下述问题:

1.1 `collect_trajectories` 实现部分

1. `collect_trajectories` 与普通 rollout 采样方式相比, 数据收集的停止条件分别是什么?

回答: 普通 rollout 的采样更像是“按步数截断”。在现有代码中, 普通采样走的是 `BaseAlgorithm.interact_with_envs()`, 它会令每个并行环境都交互 `interact_per_epoch` 步, 然后直接返回这一批 transition。因此它的主要停止条件是固定的采样步数达到上限, 而不是一定等到 episode 结束。即使某个环境中途结束, 向量化环境也会自动进入下一轮 episode, rollout 仍然继续把当前 epoch 需要的步数收满。

`collect_trajectories` 的停止条件则是“按轨迹条数截断”。它使用的是 `TrajectoryRollout`, 外层循环条件是 `while not self.traj_rollout.full`, 也就是直到缓冲区中已经保存了 `trajnum` 条轨迹才停止。每一条轨迹内部再以环境返回终止信号或达到 `max_episode_length` 为结束条件。因此 trajectory 模式关心的是收集够若干条相对完整的轨迹, 而普通 rollout 模式关心的是收集够固定数量的交互步。

2. 在 trajectory 采样模式下, 每一条轨迹需要保存哪些信息? 这些信息分别在后续 TRPO 更新中起什么作用?, 当环境返回终止信号时, 轨迹又会如何结束? 未达到最大长度的部分又应如何处理?

回答: 在 trajectory 模式下, 每条轨迹至少要保存 `states`、`actions`、`rewards`、`done`s 和 `log_probs`。这也正好对应 `TrajectoryRollout.add_traj()` 中读入的几个字段。

其中, `states` 用来输入 actor 和 critic: actor 需要它重新计算当前策略下的动作概率, critic 需要它估计状态价值 $V(s_t)$ 。`actions` 是当时实际采样到的动作, 后续计算 $\log \pi_{\theta}(a_t|s_t)$ 时必须用同一个动作。`rewards` 用来从后往前计算 return, 也就是每个时间步的折扣回报。`done`s 表示这个 transition 后 episode 是否真正结束, 计算 return 或 GAE 时需要用它切断递推, 例如终止后不应该再把后面状态的价值接到当前回报上。`log_probs` 保存的是旧策略采样动作时的对数概率, 在 TRPO 更新 actor 时会和新策略的 log probability 相减, 得到重要性采样比率

$$\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} = \exp(\log \pi_{\theta}(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t)).$$

这个比率再乘上 advantage, 就是 TRPO surrogate objective 的核心部分。

如果环境返回终止信号, 当前轨迹应当立刻结束, 并把已经收集到的真实步数通过 `add_traj()` 写入 `TrajectoryRollout`。没有达到 `max_episode_length` 的剩余位置不能当成真实样本使用。代码里的处理方式是这些位置保持初始化值: 状态、动作、奖励和旧 log probability 为 0, `done`s 默认为 1, `masks` 为 0。这样后面计算和展平数据时, 可以只取 `masks==1` 的部分, 避免把 padding 当成环境里真的发生过的 transition。

3. TrajectoryRollout 中的 masks 变量有什么作用? 如果不使用 masks, 计算 return 或 advantage 时可能出现什么问题?

回答: masks 的作用是标记一条轨迹里哪些位置是真实采样得到的 transition, 哪些只是为了凑齐二维数组形状而填充出来的空位。因为 TrajectoryRollout 预先分配的数组形状是 (trajnum, max_episode_length, ...), 但真实 episode 的长度通常不完全一样, 所以短轨迹后面一定会有 padding。add_traj() 中的 self.masks[self.pos,:traj_length] = 1 就是在记录这条轨迹的真实长度。

如果不用 masks, 计算 return 或 advantage 时很容易把 padding 部分也算进去。轻一点的问题是样本数量被虚假放大, return、advantage 的均值和方差都会被一堆 0 奖励、0 状态污染, advantage normalization 也会跟着偏掉。更严重的问题是, 递推计算

$$R_t = r_t + \gamma(1 - d_t)R_{t+1}$$

或 GAE 时, 可能会把无效时间步和真实时间步混在一起, 使一条短轨迹后面的空白部分参与 critic 训练或 actor 的 surrogate loss。这样策略更新看起来用了很多数据, 实际上有一部分是伪造的 padding, 会让梯度方向变得不可靠。

4. collect_trajectories 在收集完一批轨迹后为什么需要重新初始化环境状态?

回答: 收集完一批 trajectory 之后调用 self.initialize(), 本质上是把下一次采样的起点重新放回环境 reset 后的初始分布。trajectory 模式在一个 epoch 内会不断推进环境, 可能有的环境刚好终止, 有的环境只是因为缓冲区已经满了而停在 episode 中间。如果不重新初始化, 下一轮收集会从这些残留状态继续开始, 第一条轨迹就不一定是真正从 episode 起点开始的轨迹, 轨迹边界也会变得不干净。

对 TRPO 这种 on-policy 方法来说, 每次更新后旧数据都会被清空, 下一批数据应该由更新后的当前策略重新采样。重新初始化环境可以让“采样一批轨迹、用这一批更新、清空缓冲区”这个流程更清楚, 也避免上一批未结束的环境状态和下一批新策略采样混在一起。这样做会牺牲一点连续交互的样本利用率, 但换来的是更简单、更稳定的轨迹组织方式。

1.2 __update_policy 实现部分

该部分理论主要依赖于 Trust Region Policy Optimization 中的 Theorem 1, 代码实现参考 <https://spinningup.openai.com/en/latest/algorithms/trpo.html> 中的 Algorithm 1:

1. 解释该公式中每一项的实际意义, 这个公式在什么情况下可以保证策略的单调改进? 由此直观解释 TRPO 算法的设计思路 (优化目标和约束的设计? 先概述你的大致想法, 具体细节在后续提问中体现)。

回答: TRPO 的 Theorem 1 可以粗略理解成:

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - C D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})^2.$$

这里 $\eta(\pi_{\text{new}})$ 是新策略真正能得到的期望折扣回报, 也就是我们最想提升但又很难直接精确优化的目标。 $L_{\pi_{\text{old}}}(\pi_{\text{new}})$ 是用旧策略采样到的状态分布近似评价新策略的代理目标, 可以看成“如果状态分布暂时不变, 只把动作策略换成新策略, 新策略大概能好多少”。其中会用到旧策略下的 advantage:

$$A_{\pi_{\text{old}}}(s, a) = Q_{\pi_{\text{old}}}(s, a) - V_{\pi_{\text{old}}}(s),$$

它表示在状态 s 采取动作 a 比旧策略平均动作好多少。

后面的惩罚项 $C D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})^2$ 用来估计“新旧策略差得太多”带来的风险。 $D_{\text{TV}}^{\max}$ 是所有状态上新旧动作分布的最大 TV 距离; C 和 advantage 的最大幅度、折扣因子 γ 有关, 直观上环境越长、advantage 波动越大, 大步更新的风险就越大。

所以只要新策略让代理目标的提升量大于这个风险惩罚项, 定理下界就会上升, 从而可以保证策略性能单调改进。由此 TRPO 的思路就是: 一方面最大化 surrogate objective, 让新策略更多选择旧策略看来 advantage 为正的的动作; 另一方面限制新旧策略距离, 避免策略一步改太猛导致真实状态分布也大幅改变。也就是说, TRPO 不是简单地沿策略梯度走一大步, 而是在一个“可信的小范围”内尽量找更好的策略。

2. 和文章 Approximately optimal approximate reinforcement learning 中的 Theorem 4.1 (即 TRPO 论文中的公式 (6)) 作对比, TRPO 作者在分析上的处理技术有什么具体的不同, 使得 TRPO 可以推广到所有一般的随机策略类而不是给定的混合策略类?

回答: Kakade 和 Langford 的 Theorem 4.1 分析的是一种比较特殊的更新: 新策略写成

$$\pi_{\text{new}} = (1 - \alpha)\pi_{\text{old}} + \alpha\pi',$$

也就是在每个状态以 $1 - \alpha$ 的概率继续用旧策略，以 α 的概率改用另一个候选策略。这种形式的好处是分析很方便，因为每一步“偏离旧策略”的概率就是显式的 α ，但它限制了策略更新必须属于这种混合策略类。

TRPO 的处理更一般。它不要求新策略一定写成旧策略和某个策略的显式混合，而是用新旧策略在每个状态上的距离来控制它们生成轨迹时发生差异的概率。具体来说，如果

$$\alpha = \max_s D_{\text{TV}}(\pi_{\text{old}}(\cdot|s), \pi_{\text{new}}(\cdot|s)),$$

那么可以把“新旧策略在某一步采到不同动作”的概率用 α 控住，再把多步轨迹中状态分布的偏移累积起来。这样分析对象就从“给定的混合策略”变成了“任意两个随机策略之间的分布距离”，所以可以推广到神经网络参数化的随机策略。

3. TRPO 中策略更新的 surrogate objective $L_{\pi_{\text{old}}}(\pi_{\text{new}})$ 一项在实际算法中是如何设计的？

回答：实际算法里无法对所有状态和动作求精确期望，所以用旧策略采样得到的一批数据做 Monte Carlo 估计。代码中的核心形式是

$$\hat{L}(\theta) = \frac{1}{N} \sum_{t=1}^N \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t.$$

这里 (s_t, a_t) 来自采样缓冲区， $\hat{A}_t$ 是由 return 和 critic 估计出来的 advantage。比率

$$\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

是重要性采样修正，因为数据是旧策略采的，但我们要评价当前候选新策略。

对应到代码里，就是先保存采样时的 `old_log_probs`，更新时重新计算 `new_log_probs = self.agent.log_prob(states, actions)`，再令

$$\text{ratio} = \exp(\text{new_log_probs} - \text{old_log_probs}),$$

最后用 `surrogate = (ratio * advantages).mean()` 作为 actor 想要最大化的目标。如果开启 `advan_norm`，代码还会先把 advantage 标准化，这不会改变“正 advantage 鼓励、负 advantage 抑制”的方向，但能让数值尺度更稳定。

4. 为什么 TRPO 需要约束新旧策略之间的 KL divergence 而不是定理中的 TV 距离？在实际算法中我们采用 KL 散度的二阶近似 $D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta}) \sim \frac{1}{2} \Delta \theta^T (\nabla_{\theta}^2 D_{\text{KL}}) \Delta \theta$ ，延续该思路解释为什么在 Algorithm 1 中我们会采用 line search 搜索参数 (Algorithm 1 line 8&9)？

回答：理论里用 TV 距离比较自然，但在神经网络策略上直接优化 TV 距离不方便：它不如 KL 散度容易求导，也不容易和高斯策略、Categorical 策略的分布形式配合。KL divergence 通常有现成的解析式或稳定的自动微分实现，而且 Pinsker 不等式说明 TV 距离可以被 KL 散度控制，所以实际算法中就用 KL 作为更好算的替代约束。

在当前代码中，KL 约束对应

$$D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta}) \approx \frac{1}{2} \Delta \theta^T H \Delta \theta,$$

其中 H 是 KL 对策略参数的 Hessian，也可以理解成 Fisher 信息矩阵。因此 TRPO 先用共轭梯度近似求解

$$Hx = g,$$

得到自然梯度方向 $x \approx H^{-1}g$ ，再根据 $\frac{1}{2} \Delta \theta^T H \Delta \theta \leq \delta$ 算出理论步长。

但这个步长只是局部二阶近似下的结果，实际神经网络是非线性的，advantage 和 KL 也都是用有限样本估计的，共轭梯度本身也只是近似求解。所以一步走到理论步长时，真实计算出来的 KL 可能超过 δ ，surrogate objective 也可能没有提升。这就是 Algorithm 1 要做 line search 的原因：沿着同一个自然梯度方向，从较大的步长开始尝试，如果候选参数同时满足

$$\hat{L}(\theta_{\text{new}}) > \hat{L}(\theta_{\text{old}}) \quad \text{且} \quad \hat{D}_{\text{KL}}(\pi_{\theta_{\text{old}}}, \pi_{\theta_{\text{new}}}) \leq \delta,$$

就接受；否则把步长乘上 `linesearch_coefficient` 继续回退。对应代码就是 `candidate_params = old_params + alpha * stepsize * gradient_direction`，然后检查 `ls_surrogate > surrogate` 和 `ls_kl <= self.delta`。

2 探索部分

请从以下五个环境中至少选择三个进行测试: **Humanoid-v5**, **Ant-v5**, **HalfCheetah-v5**, **Hopper-v5**, **Walker2d-v5** 并思考下述问题:

1. 在相同超参数设置下, 三个环境的训练难度是否一致? 请结合 reward 曲线进行分析。
2. 哪个环境中 trajectory 与 rollout 的样本利用率差异最明显? 可能的原因是什么?
3. 在 trajectory 模式和 rollout 模式下, advantage 与 return 的计算方式有什么不同?

2.1 Advantage Normalization 的影响

1. 请分别设置 `advan_norm=true` 和 `advan_norm=false` 进行实验。两组实验的平均回报曲线有何差异?
2. 加入 advantage normalization 后, 训练过程是否更加稳定? 请结合 reward 曲线、KL divergence 或 surrogate objective 的变化进行分析。
3. 在你选择的测试环境中, advantage normalization 是否总是带来性能提升? 如果不是, 可能的原因是什么?

2.2 Trajectory 与 Rollout 的样本利用率比较

1. 在 rollout 模式下, 每次策略更新实际使用了多少个 transition? 在 trajectory 模式下, 每次策略更新又实际使用了多少个有效 transition?
2. 对比 trajectory 与 rollout 两种模式, 哪一种样本利用率更高?
3. 样本利用率更高是否一定意味着训练效果更好? 请结合实验曲线进行分析。

3 总结与反思

1. 在本项目中, `collect_trajectories` 的实现是否会改变 TRPO 的 on-policy 特性? 说出你的判断理由。
2. 如果只能保留一种采样方式, 你会选择 trajectory 还是 rollout? 请结合样本利用率和训练性能说明理由。
3. 完成实验报告的撰写后, 你对 TRPO 算法是否有了新的感悟或认识? (**这个问题不建议 AI 辅助生成, AI 时代最珍贵的是人类自我的想法。相信你写完报告后总会有一些自己的思考, 或许是疑惑或许是感悟, 完善与否不重要, 写出你自己的想法就好。**)
4. 针对本次算法实现, 你是否有什么想法和想改进的建议, 你在实现过程中有没有因为项目 README.md 或者实验报告引导和介绍问题导致卡了很久, 如果有, 请详细说明, 并给出你的建议。关于代码/实验报告问题难度你是否有感觉很难/简单?